

LLMProvider

LLMProvider

Бір интерфейс, 43 провайдер — схемалық қорғағыштар, қайталаулар және денсаулық мониторингі кіріктірілген.

Қысқаша мазмұны

LLMProvider — жалпы қолданылатын, қайта пайдаланылатын Go модулі. Ол біріккен LLMProvider интерфейсін және оған байланысты өндірістік төзімділік үлгілерін анықтайды: схемалық қорғағыш, денсаулық мониторингі, кері шегінумен қайталау, кейін жүктеу. Сонымен қатар, бір келісімшарт арқылы 43 нақты провайдердің іске асыруын ұсынады, OpenAI-ке үйлесімді жалпылама адаптер және адал, қатты кодталмаған резервтік модельді анықтау жүйесі бар.

Қысқаша сипаттама

Қайта пайдаланылатын Go модулі бір LLMProvider интерфейсін (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) және ақауға төзімділік механизмдерін ұсынады: схемалық қорғағыш, денсаулық монитормен, кері шегінумен қайталау, кейін жүктеу. 43 провайдер адаптері және OpenAI-ке үйлесімді жалпылама адаптермен бірге. Өзара әрекеттесуге қауіпсіз.

Толық сипаттама

LLMProvider — әрбір LLM-ті пайдаланатын қызметке қажет, бірақ ешкім дұрыс жасай алмайтын абстракциялық қабат. Ол нақты демо мен нақты трафикпен кездескенде тірі қалатын жүйені бөліп тұратын қарапайым құбырларды қамтиды. Бір ғана мүмкіндіктерге негізделген интерфейсін анықтайды — Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig — осылайша қолданба коды 43 бекендтің қайсысы сұрауды қабылдағанына қарамастан бір ғана

келісімшартқа бағытталады. Содан кейін ол өндірістік ортада сенімсіз провайдер шақыруларын өміршең жүйеге айналдыратын операциялық қатаңдықты ұсынады. Үш күйлі схемалық қорғағыш (жабық → ашық → жартылай ашық) кез келген провайдерді — *оның ағындық арнасын қоса алғанда* — мөлдір түрде орап алады, мұнда бос ағын сәтсіздік ретінде дұрыс есептеледі. Бір ғана дұрыс жұмыс істемейтін бекенд бүкіл қызметті тоқтата алады. Орталық CircuitBreakerManager барлық қорғағыштарды бірден бақылайды. Реттелетін денсаулық мониторы провайдерлерді белгілі бір уақыт аралығындағы тексерулер арқылы сау / төмендетілген / ауру / белгісіз күйлерге бөледі, осылайша төмендеу апатқа ұшырағанға дейін байқалады. Қайталау логикасы кері шегінуді әртүрлі кідірістермен біріктіреді, мәнді шешімдер қабылдайды: қайталауға тұрарлық қателерді (429, 5xx, уақытша желілік ақаулар) қайталайды, 4xx немесе тоқтатылған контексті қайталауға тырыспайды. Кідірістер шектеледі, сондықтан кері шегіну дауылы қашып кетпейді. Ал кейін жүктеу үлгісі әрбір провайдердің құрылымын оның алғашқы нақты қолданысына дейін кейінге қалдырады — бұл әдейі жасалған шешім, ол 43 провайдердің барлығын тіркеуді тегін етеді.

Модуль 43 нақты провайдер пакетін және generic OpenAI-ке үйлесімді адаптерді ұсынады, ол толық интерфейсті кез келген `/v1/chat/completions` соңына қарсы іске асырады — Bearer аутентификациясы, SSE ағыны дұрыс [DONE] өңдеуімен. Арнайы пакеті жоқ вендор да сіз адаптерді оның URL-не бағыттаған сәттен бастап бірінші дәрежелі азамат болып саналады. Мәнмәтіндер бір ғана жерде (аріkeys, қатаң `ApiKey_<Provider>` келісімі бойынша) шешіледі, бұл “қатты кодталған кілт сынақ жинағын өткізеді, нақты кілт ешқашан қосылмаған, өнім өндірісте істен шығады” деген қателерді алдын алады. Модельді анықтау әдейі, тіпті қатаң түрде адал: ол TTL кәші арқылы тірі провайдер API-ларын сұрайды, ал басқару саясаты бойынша ескі қатты кодталған резервтік деңгей толығымен жойылды. Тірі анықтау сәтсіз болған кезде LLMProvider ештеңе қайтармайды, ескі каталогты емес, сондықтан шақырушыға шақырылмайтын модель идентификаторы берілмейді. Бұл барлығы өзара әрекеттесу үшін ағынға қауіпсіз құрылған.

Неліктен құрастырдық

Naive LLM шақырулары өндірісте істемейді — провайдерлер шектеу қояды, сапасын төмендетеді немесе жұмыс істемей қалады, ал бір дұрыс емес бэкэнд бүкіл қызметті құлатып жібереді. Модель каталогтары ауытқиды, ал қатаң кодталған тізімдер шақырушыларға енді жұмыс істемейтін ID-ларды

береді. LLMProvider интерфейсті, төзімділік үлгілерін және ақиқатты ашуды орталықтандырады, осылайша әрбір тұтынушы ақауға төзімділік пен шынайылықты тегін алады.

Неліктен бұл ойынды өзгертеді

Бұл «LLM провайдерін біріктіруді» бір ғана әрекетке айналдырады — бір интерфейсті жүзеге асыру немесе жалпы адаптерді соңғы нүктеге бағыттау жеткілікті, содан кейін ол провайдерді автоматты түрде және ашық түрде схема бұзушы, денсаулықты бақылау және кездейсоқ кері шегіну қайталаумен қаптайды. Төзімділік енді әр команда өздігінен қайта ойлап табуға тиіс нәрсе (жаман, мерзімнен тыс, алғашқы ақаудан кейін) болмай, кітапхананың барлық 43 бэкэндтегі әдепкі мінез-құлқына айналады. Сенімділік инженериясы бір рет жазылып, қатаң сынақтан өтеді және оны импорттаған әркім тегін алады.

Инновациялық жақтары

- **Бір қабілетті интерфейс** — аяқтау, ағынды жіберу, денсаулықты тексеру, қабілеттер мен конфигурацияны тексеру бір келісімшартқа біріктірілген, оны әр бэкэнд бірдей қастерлейді.
- **Ағынды жіберуді қоса алғанда, ашық схема бұзушы қаптау.** Бұзушы CompleteStream арнасын қорғайды, тек сұраныс/жауапты ғана емес, ал бос ағынды нақты ақау ретінде қабылдайды — өлімшілдіксіз, құлыпсыз тыңдаушыларды хабарлаумен.
- **43 провайдерлік пакет + жалпы OpenAI-ке үйлесімді адаптер** — жеке пакеттер жұқа болып қалады, ал /v1/chat/completions сөйлейтін тізімде жоқ кез келген сатушы адаптерді оған бағыттаған сәтте жұмыс істейді.
- **Бір аккредиттеу органы (apikeys)** — ApiKey_<Provider> орталық айнымалыларын тек бір жерден оқиды, бұл құрылымдық түрде «жасыл сынақтар, бұзылған өнім» сәйкессіздігін жойып, тек ескерту жасаудың орнына.
- **Шынайы модель ашу (қатаң кодталған резервтік нұсқасы жоқ)** — TTL кәші арқылы тірі провайдерлік API-лар; ақау болған жағдайда nil қайтарады, ешқашан ескі немесе жасанды каталог бермейді, ол шақырылмайтын ID-ларды таратады.
- **sync.Once арқылы кейінге қалдырылған инициализация** — құрылым бірінші қолдануға дейін кейінге қалдырылады, сондықтан барлық 43 провайдерді тіркеу сіз олардың біреуін шақырғанға дейін шығынсыз болады.
- **Алдауға қарсы, көптілді Challenge стегі** — бес аймақта схема, денсаулық және қайталау мінез-құлқын тексеріп

шығатын нақты жүгіруші, жұп мутациялық тестілеумен шектелген (мутациясыз код нөлден шығуы керек; енгізілген мутация 99-дан шығуға мәжбүр етуі керек), сондықтан өткен топтама нақты жұмыс істеу мінез-құлқын дәлелдейді.

Ең үлкен техникалық қиындықтар және оларды қалай шештік

- **Провайдерлердің тізбекті ақаулары.** Бір тұрақсыз бэкэнд бүкіл қызметті құлатып жібермеуі керек. Бұл үш күйлі схема бұзушымен шешілді (жабық → ашық → жартылай ашық), ол кез келген провайдерді *және оның ағындысын* ашық түрде қаптайды, тұрақты ақау кезінде ашық күйге ауысады, жартылай ашық күйде қалпына келуді тексереді және `CircuitBreakerManager` арқылы орталықтандырылған.
- **Уақытша қателер мен шектеулер.** Бұл күйге байланысты экспоненциалды кері шегіну мен кездейсоқ ауытқумен шешілді — $\min(\text{БастапқыКідіріс} \cdot \text{Көбейткіш}^{(n-1)}, \text{ЕңҮлкенКідіріс}) \pm \text{ауытқу}$ — қайталаулар бір-бірімен синхрондалып, «көпшілік шабуылға» айналмайтындай етіп таралады. Ол тек қайталау керек нәрсені қайталайды (429, 500, 502, 503, 504 және желі қателері) және жойылған контекст немесе басқа 4xx үшін әрекеттерді бекерге шығармайды.
- **Тіркелген бірақ пайдаланылмаған көптеген провайдерлерді масштабтау.** 43 провайдер тіркелгенімен, кез келген қызметте олардың тек бірнешеуі ғана жұмыс істейді, ал ертерек инициализация тек шығын болар еді. Бұл `sync.Once` арқылы қорғалған кейінге қалдырылған инициализациямен шешілді, сондықтан сіз шақырған провайдерлер ғана өз құрылымын төлейді.
- **Жарамсыз модель ID-ларын беру.** Бұл CONST-036 бойынша ашудың қатаң кодталған резервтік деңгейін толығымен жоюмен және тірі ашу ақауында ештеңе қайтармаумен шешілді — сонымен қатар қорғаушылық көшірме қайтару арқылы шақырушы кәшті өзгерте алмайды немесе басқа оқырманмен жарыса алмайды. Шынайылық құрылымдық түрде қадағаланады, тек келісіммен емес.
- **Ағынды жіберу + параллельдік дұрыстығы.** Ақаудың нәзік түрі — бұзушының құлыпы мен тыңдаушыларды шақырудың өлімшілігі. Бұл тыңдаушыларды көшірме алып, оларды құлыпсыз 5 секундтық уақыт шегінде хабарлаумен және қалпына келтіру кезінде құлыпты ашумен шешілді — әрбір компонент параллельдік қолдануға арналған және -race топтамасымен бекіті

Технологиялық стек

- **Go (1.25.3)** — бірінші дәрежелі параллелділік, статикалық бинарлық файлдар және мықты стандартты кітапхана үшін таңдалған; модульді, интерфейсті, барлық төзімділік примитивтерін және 43 адаптерді қамтиды.
- **net/http (stdlib)** — әдейі тәуелсіз HTTP: әрбір провайдер клиенттерін, жалпы OpenAI-ке сәйкес адаптерді және тікелей ашу шақыруларын қуаттайды, сондықтан тексеру немесе жаңарту үшін үшінші тараптық тасымалдаушы жоқ.
- **logrus** — құрылымдық, деңгейлі журналдау операторларға көрінетін жерлерде: схема бұзушының күй өзгерістерінде және ашу жолында.
- **testify** — сынақ жиынтығын және ең маңыздысы — мутациялық тармақтарды бекітуді басқарады, бұл жасыл жүгірудің мағынасы бар екенін білдіреді.
- **yaml.v3** — халықаралықтандыру бандлдерін және конфигурацияны адамдардың түзетуіне ыңғайлы форматта талдайды.
- **digital.vasic.models** — ортақ LLMRequest / LLMResponse / ProviderCapabilities түрлері, барлық адаптер бірдей сөздікпен сөйлеуі үшін бір жерде сақталған (құжатталған орындау уақыты тәуелділігі).
- **Бірінші тараптық пакеттер** — circuit, health, retry, apikeys, discovery, providers/ (43 вендор + generic), және i18n: төзімділік пен интеграциялық бетті шағын, жеке сынақталатын бірліктерге бөліп, бір монолитке біріктірмейді.
- **.env + ~/api_keys.sh (ApiKey_<Provider> үлгісі)** — аккредитацияның бірден-бір анық көзі, сондықтан кілттер сынақтарда да, өндірісте де бірдей әдіспен қосылады.
- **Makefile race suite (-race -p 1) + Challenge runner** — алда тұрған жалғандыққа қарсы негіз: параллелділік дұрыстығын жарыс детекторы дәлелдейді, ал Challenge runner нақты мінез-құлықты аласапыран, ddos, масштабтау, стресс, тікелей ашу және тоқтаусыз сценарийлер арқылы тексереді.

Жағдай және ақиқат жазбалары

- **Жағдайы: бета.** Ажыратылған қайта пайдаланылатын модуль; GitHub репозиторийі ашық.
- **Лицензиясы: анықталмаған.** Біркелкі емес — doc.go MIT дейді, ал Apache-2.0 үлгісіндегі LICENSE файлы бар — жарияламас бұрын тексеріңіз.
- **LLMsVerifier** — үлгілер каталогының ресми көзі үшін жоғарыдағы бірден-бір ақиқат көзі. helix-deps.yaml

манифесті ескірген (құжаттама `digital.vasic.models`-ке тәуелділікті көрсетеді, ал `deps: []` деп жариялайды); ашудың “Tier 2 (models.dev)” жоспарланған бос орын, белсенді емес.

Басымдық деңгейі: Helix-негізгі (LLM-инфрақұрылым кластері — ажыратылған қайта пайдаланылатын модуль). HelixTrack-нан кейін орналасады.